COMPUTATIONAL FINANCE

Jacopo Zavalloni

July 2025

1 Introduction

In this project, we implement a Monte Carlo pricing framework based on the geometric version of the Scott stochastic volatility model. The objective is to simulate the asset dynamics, estimate European Put and Call option prices, and compute the corresponding implied volatilities. We firstly compare our model-based results with benchmark market data provided by the instructor and then evaluate results with different model parameters. The report is structured in two main parts. First (from Section 2 to 4), we define the theoretical framework and detail all the validation tests used to ensure the correctness and consistency of the simulation engine. Second (Section 5), we outline the empirical approach adopted in the numerical implementation, explaining the steps followed, the rationale behind the simulation strategies, and finall we present and discuss the results obtained from the model.

Model description. In the geometric formulation of the Scott stochastic volatility model, the asset price S_t follows a log-normal diffusion process with stochastic volatility. The model dynamics under the risk-neutral measure are given by:

$$\begin{aligned} dS_t &= \sqrt{m} \cdot e^{y_t/2} \cdot S_t \, dW_t, \\ dy_t &= -\gamma y_t \, dt + \eta \, dZ_t, \\ \sigma_t &= \sqrt{m} \cdot e^{y_t/2}, \end{aligned}$$

- y_t is an Ornstein–Uhlenbeck process describing the log-volatility;
- σ_t is the instantaneous volatility of S_t;
- W_t and Z_t are Brownian motions with correlation $\mathbb{E}[dW_t dZ_t] = \rho \, dt$.

2 Simulation of a random trajectory of S_t

Let us derive a numerical scheme to simulate a full trajectory $\{S_n\}_{n=0}^N$ over the interval $[0, T]$ using a time step $\Delta t = T/N$. The approach is based on the following steps:

Discretization of the volatility process y_t The process y_t follows an Ornstein–Uhlenbeck dynamic, so we can simulate the process exactly over each time step. The SDE has an exact solution and so an exact update rule given by (see Appendix):

$$y_{n+1} = y_n e^{-\gamma \Delta t} + \eta \cdot \sqrt{\frac{1 - e^{-2\gamma \Delta t}}{2\gamma}} \cdot \xi_n, \quad \xi_n \sim \mathcal{N}(0,1)$$

Generation of correlated Brownian increments Using Gaussian Bivariate Decomposition (Cholesky decomposition) we can simulate dW_n and dZ_n with the prescribed correlation ρ. Thus starting from two independent Normal Random Variables we define:

$$\begin{aligned} dZ_n &= \sqrt{\Delta t} \cdot \xi_n^{(1)}, \\ dW_n &= \sqrt{\Delta t} \cdot \left(\rho \xi_n^{(1)} + \sqrt{1 - \rho^2} \cdot \xi_n^{(2)} \right), \end{aligned}$$

with $\xi_n^{(2)} \sim \mathcal{N}(0,1)$ independent of $\xi_n^{(1)} \sim \mathcal{N}(0,1)$. This construction guarantees:

$$\mathbb{E}[dW_n \cdot dZ_n] = \rho \Delta t.$$

Simulation of S_t via Euler scheme We rewrite the SDE for S_t using Itô's lemma:

$$d \ln S_t = -\frac{1}{2} \sigma_t^2 \, dt + \sigma_t \, dW_t, \qquad \text{where } \sigma_t = \sqrt{m} \cdot e^{y_t/2}.$$

Discretizing this yields the log-Euler scheme:

$$\ln S_{n+1} = \ln S_n - \frac{1}{2} \sigma_n^2 \Delta t + \sigma_n \cdot dW_n,$$

or equivalently, in exponential form:

$$S_{n+1} = S_n \cdot \exp\left(-\frac{1}{2} \sigma_n^2 \Delta t + \sigma_n \cdot dW_n \right).$$

Full trajectory generation Combining the above steps, a full random trajectory $\{S_n\}$ is simulated by initializing:

$$S_0 = S_0, \quad y_0 = 0,$$

and iteratively applying:

$$\begin{aligned} y_{n+1} &= y_n - \gamma y_n \Delta t + \eta \sqrt{\Delta t} \, \xi_n^{(1)}, \\ dW_n &= \sqrt{\Delta t} \cdot \left(\rho \xi_n^{(1)} + \sqrt{1 - \rho^2} \cdot \xi_n^{(2)} \right), \\ \sigma_n &= \sqrt{m} \cdot e^{y_n/2}, \\ S_{n+1} &= S_n \cdot \exp\left(-\frac{1}{2} \sigma_n^2 \Delta t + \sigma_n \cdot dW_n \right). \end{aligned}$$

3 Option Pricing and Implied Volatility Computation

After simulating a large number of asset price trajectories S_t under the geometric Scott model, we compute the price of a European option via Monte Carlo. Specifically, we estimate the fair price of a put (or call) as:

$$P = \mathbb{E}[\max(K - S_T, 0)] \quad \text{or} \quad C = \mathbb{E}[\max(S_T - K, 0)],$$

where the expectation is approximated by averaging over the simulated terminal prices S_T.

$$P_{\text{MC}}(K, T) = \frac{1}{M} \sum_{m=1}^{M} (K - S_T^{(m)})^+$$

Alongside the estimated option price, we also compute the standard error associated with the Monte Carlo estimator. We evaluate the Monte Carlo error as:

$$\hat{\sigma}_{\text{MC}} = \frac{\hat{\sigma}_{\text{payoff}}}{\sqrt{M}},$$

where $\hat{\sigma}_{\text{payoff}}$ is the empirical standard deviation of the option payoffs (either $(K - S_T)^+$ for puts or $(S_T - K)^+$ for calls). This standard error provides a confidence band around the Monte Carlo price $P_{\text{MC}} \pm \hat{\sigma}_{\text{MC}}$.

We then compute the implied volatility σ_{impl}, defined as the volatility parameter which, when used in the Black–Scholes pricing formula, yields the same option price:

$$P_{\text{BS}}(S_0, K, T, \sigma_{\text{impl}}) = P_{\text{MC}}.$$

This is achieved numerically by solving the nonlinear equation:

$$f(\sigma) = C_{\text{BS}}(S_0, K, T, \sigma) - C_{\text{MC}} = 0,$$

using numerical algorithm such as bisection scheme and Brent's method. (see Appendix)

$$P_{\text{BS}}(S_0, K, T, \sigma) = K\,\Phi(-d_2) - S_0\,\Phi(-d_1)$$
$$d_1 = \frac{\ln(S_0/K) + \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}$$

4 Numerical consistency and Validation

To ensure the correctness of the Monte Carlo simulation, we perform several checks:

Martingality of the simulated asset process

We verify that the simulated asset price process S_t satisfies the martingale property under the risk-neutral measure. Specifically, we check that:

$$\mathbb{E}[S_{n+1} \mid \mathcal{F}_n] = S_n.$$

$$\mathbb{E}[S_t] \approx S_0 \quad \forall t \in [0, T]$$

In our implementation, the test is performed as follows:

- The empirical mean of S_t is computed at each time step across the Monte Carlo trajectories.

$$\hat{\mu}_t := \frac{1}{M} \sum_{i=1}^{M} S_t^{(i)}.$$

- The standard error of the mean is estimated as:

$$\widehat{\mathrm{StdErr}(S_t)} = \frac{\widehat{\mathrm{StdDev}(S_t)}}{\sqrt{M}}$$

  where M is the number of simulated paths.

- A point-wise confidence band is constructed around the initial value S_0, using $\pm 2 * \widehat{\mathrm{StdErr}(S_t)}$.
- A plot is produced showing $\mathbb{E}[S_t]$, the confidence band, and the constant line at S_0. Any time step where $\mathbb{E}[S_t]$ deviates beyond the band is marked in red.

Verification of Brownian correlation

We have simulated the Brownian increments dW_n and dZ_n using the standard Cholesky construction so we then empirically verify that:

$$\widehat{\mathbb{E}}[dW_n \cdot dZ_n] \approx \rho \cdot \Delta t,$$

which ensures the theoretical correlation structure $\mathbb{E}[dW_t \cdot dZ_t] = \rho\, dt$ is respected in the discretized setting. During the Monte Carlo simulation of S_t, we store all generated increments dW_n and dZ_n across all paths and time steps. Then the full arrays of Brownian increments dW and dZ are flattened into vectors of size $M \cdot N$. After the simulation, we compute the empirical covariance and correlation:

$$\widehat{\mathrm{Cov}}(dW_n, dZ_n) = \frac{1}{MN} \sum_{i=1}^{M} \sum_{n=1}^{N} dW_n^{(i)} \cdot dZ_n^{(i)},$$

$$\widehat{\mathrm{Corr}}(dW_n, dZ_n) = \frac{\widehat{\mathrm{Cov}}(dW_n, dZ_n)}{\hat{\sigma}_{dW} \cdot \hat{\sigma}_{dZ}}.$$

The empirical covariance is then compared to the theoretical value $\rho \cdot \Delta t$. Checking whether:

- The empirical correlation matches the target ρ to several decimal places.
- The empirical covariance matches $\rho \cdot \Delta t$ within the expected Monte Carlo tolerance: if the absolute error remains smaller than $5 \cdot \mathcal{O}(1/\sqrt{MN})$.
- The theoretical value $\rho \cdot \Delta t$ is contained in the confidence level at 95% for the empirical covariance;

$$\widehat{\mathrm{Cov}} \pm 1.96 \cdot \frac{\widehat{\mathrm{StdDev}}(dW \cdot dZ)}{\sqrt{MN}}.$$

Verification of the Relationship between $\phi(t)$ and $\theta(t)$

In the geometric version of the Scott stochastic volatility model, the deterministic function $\phi(t)$ and the mean-reversion level $\theta(t)$ of the log-volatility process are related by the differential equation:

$$\theta(t) = \frac{1}{\gamma} \cdot \frac{d\phi(t)}{dt} + \phi(t)$$

In our implementation, we choose a constant $\phi(t) = \ln(m)$, where m is the baseline level of the variance. Since $\phi(t)$ is constant, its time derivative vanishes:

$$\frac{d\phi(t)}{dt} = 0 \quad \Rightarrow \quad \theta(t) = \phi(t) = \ln(m)$$

Therefore, the above relationship is trivially satisfied. A numerical verification of this identity is not meaningful in this case, as both functions are constant by construction.

Monte Carlo Error Convergence

We study how the standard error on the option price decreases as the number of simulated paths M increases. We perform a convergence experiment in which the option price and corresponding standard error are computed for increasing values of M. The results are plotted on a log-log scale to visually verify whether the standard error follows the expected theoretical behavior:

$$\text{Error}(M) = \mathcal{O}\left(\frac{1}{\sqrt{M}}\right).$$

This relation means that the statistical error of the Monte Carlo price decreases as the number of simulations M increases, and it does so at a rate proportional to $1/\sqrt{M}$. It means that doubling the number of simulations reduces the error by approximately 30%, while halving the error requires about four times as many paths. The $\mathcal{O}$ notation here expresses the order of magnitude of the convergence rate, a typical feature of Monte Carlo estimators under the Central Limit Theorem and Law of Large Number.

To visually assess whether the standard error decays as $\mathcal{O}(1/\sqrt{M})$, we include a theoretical reference curve with the same functional form. This reference is anchored at the first empirical point (M_0, Error_0), and defined as:

$$\text{Error}(M) = \text{Error}_0 \cdot \sqrt{\frac{M_0}{M}}.$$

This construction ensures that the reference curve follows the expected $1/\sqrt{M}$ decay while passing through the first observed value. When plotted in log-log scale, it appears as a straight line with slope $-1/2$, allowing for a direct visual comparison with the empirical error trend.

Benchmark Compatibility

To validate the accuracy of the Monte Carlo pricing engine, we compare the simulated option prices to benchmark prices provided by the professor. For each strike–maturity pair (K, T), we check whether the benchmark price lies within the confidence interval defined by the Monte Carlo estimate:

$$\text{Benchmark Price} \in [\text{MC Price} \pm 2 * \text{StdErr}_{\text{MC}}].$$

This compatibility check is performed separately for both call and put options. It allows us to verify whether the observed discrepancy between the benchmark and the model price can be explained by statistical uncertainty alone. As part of the validation, we compute the percentage of benchmark prices that lie within the Monte Carlo confidence interval. We also visualize this comparison using a scatter plot: the benchmark prices are plotted against the Monte Carlo prices, along with green dashed lines indicating the upper and lower confidence bounds. Points that fall outside the band highlight incompatibility.

Put-Call Parity Check

Under the assumption of zero interest rates, European option prices must satisfy the Put–Call parity relation:

$$C_{\text{MC}} - P_{\text{MC}} = S_0 - K,$$

where C_{MC} and P_{MC} denote the Monte Carlo estimates of the call and put prices, respectively. To numerically verify this identity, we reconstruct each option price via the parity relation:

$$\widehat{C}^{(\text{parity})} = P_{\text{MC}} + (S_0 - K), \quad \widehat{P}^{(\text{parity})} = C_{\text{MC}} - (S_0 - K).$$

We then compare each reconstructed price with its direct Monte Carlo counterpart and check whether the difference falls within a tolerance band of $3\hat{\sigma}$, where $\hat{\sigma}$ is the standard error of the option used for reconstruction:

$$|\widehat{C}^{(\text{parity})} - C_{\text{MC}}| \leq 3 \cdot \hat{\sigma}_{\text{Put}}, \quad |\widehat{P}^{(\text{parity})} - P_{\text{MC}}| \leq 3 \cdot \hat{\sigma}_{\text{Call}}.$$

This test accounts for the statistical uncertainty in the input option price and provides a robust numerical validation of the parity condition. We perform this check across all strike–maturity combinations and report the proportion of cases in which the parity relation is statistically not respected.

Implied Volatility Validation

We compare the implied volatilities σ_{MC} obtained by inverting the Black–Scholes formula from our simulated option prices with the benchmark implied volatilities σ_{impVol} provided in the scott benchmark dataset. For each strike–maturity pair (K, T), we solve:

$$C_{\text{BS}}(S_0, K, T, \sigma) = C_{\text{MC}} \quad \text{or} \quad P_{\text{BS}}(S_0, K, T, \sigma) = P_{\text{MC}},$$

using Bisection scheme and Brent's method. To visualize the results, we produce a scatter plot of σ_{impVol} vs σ_{MC}, with a dashed 1:1 reference line and an error band of ± 0.00001 to reflect typical market noise tolerance. Each point is color-coded by either strike or maturity. Additionally, we cross-validate the implied volatilities obtained via Biseection scheme with those computed through Brent's. For each option, we compute the absolute difference between the two estimates, and we report the percentage of cases in which the discrepancy exceeds 1%.

5 Empirical Approach

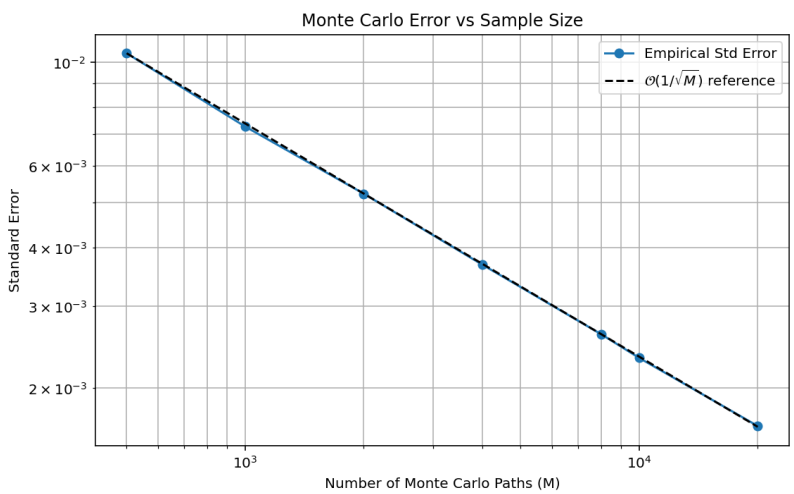
Initial setup. We begin by setting the initial parameters of the Scott model, chosen to test the model against the benchmark.

Parameter	Symbol	Value
Initial asset price	S_0	1.0
Volatility level	m	0.10
Mean reversion rate	γ	3.0
Volatility of y_t	η	0.6
Correlation	ρ	-0.6

Regarding the time discretization, we set the number of time steps per year to $N = 252$, corresponding to the number of trading days in a typical financial year. This choice ensures that the price process S_t updates daily, in line with actual market frequency.

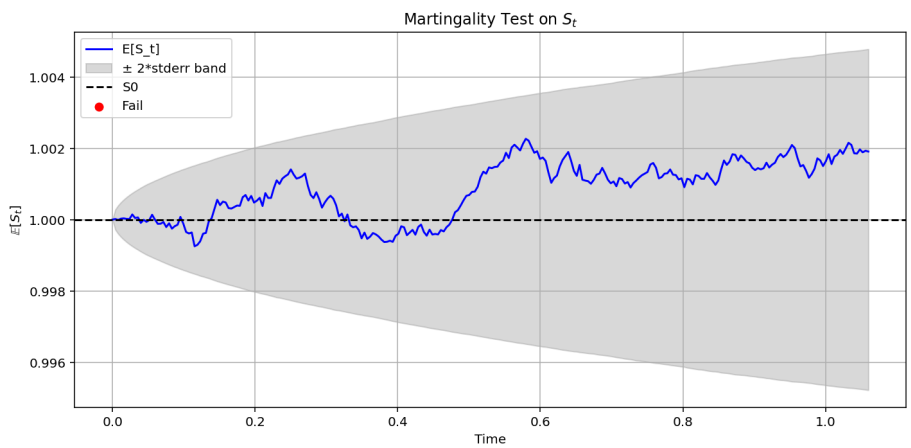
As for the number of Monte Carlo paths M, we started with a relatively low value of $M = 500$ and gradually increased it, monitoring both the martingality of the simulated process and the accuracy of the option pricing. Through iterative testing and convergence analysis, we determined that $M = 20000$ offered a good trade-off between numerical stability and computational cost, providing consistent results and stable empirical behavior of S_t.

To assess the statistical convergence of Monte Carlo pricing, we select the call option with K = 1.163 randomly chosen among those with the longest maturity and analyze the convergence of its standard error with increasing sample size. This convergence test is performed only on one option due to computational constraints and using $M_{\text{values}} = [500,\ 1000,\ 2000,\ 4000,\ 8000,\ 10000,\ 20000]$.



The strong alignment confirms that the pricing estimator satisfies the expected statistical convergence rate under the Central Limit Theorem.

Trajectory simulation. We initially tested two alternative simulation strategies: the first consisted in running a separate Monte Carlo simulation for each option maturity T, while the second involved generating a single simulation of S_t up to the maximum maturity $T_{\max}$ and truncating it at each relevant expiry. After experimentation, we found that the second method was significantly more effective. It offers greater numerical stability, since it allows us to better control the simulated trajectories and reliably test martingality and Brownian correlation. It is also much less expensive computationally and in terms of runtime, requiring only one full-path simulation regardless of the number of maturities. Finally, it provided more accurate pricing: a higher percentage of benchmark prices fell within the Monte Carlo confidence intervals. For these reasons, the empirical analysis is based on a single, unified Monte Carlo simulation strategy. Here the martingality test on the simulation:



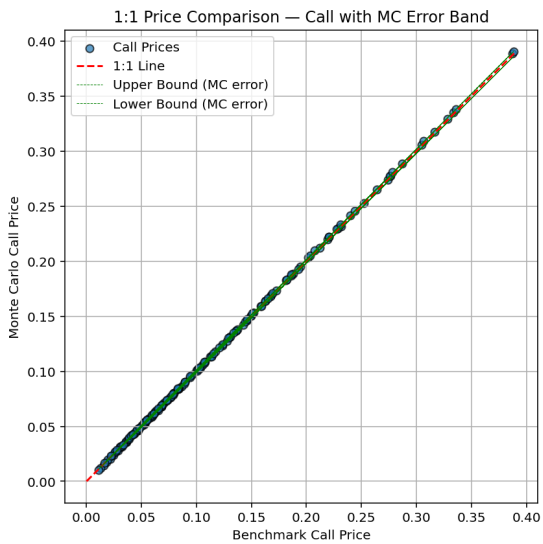
The simulated process remains within the expected range, confirming that S_t behaves as a martingale. And so here summirezes the empirical correlation analysis between the Brownian increments dW and dZ:

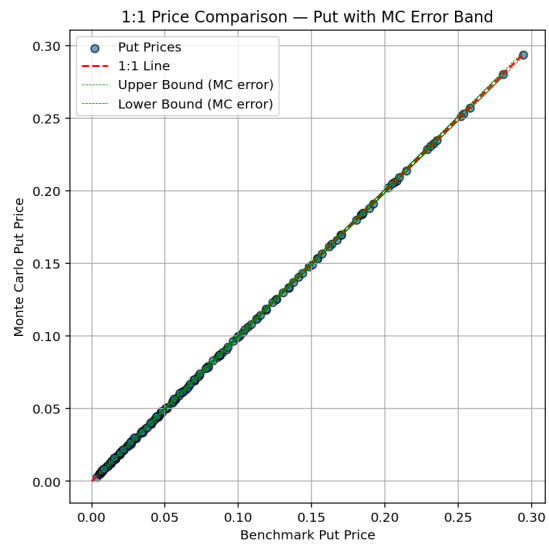
```
=== CORRELATION CHECK ===
Empirical Cov(dW, dZ):   -2.380036e-03
Theoretical Cov:         -2.382022e-03
Difference:              1.99e-06

Empirical Corr(dW, dZ):  -0.5997
Target Corr:             -0.6
```

```
--- Confidence Interval for Cov(dW, dZ) ---
95% CI: [-2.383962e-03, -2.376111e-03]
Contains theoretical cov (-2.382022e-03)? YES

--- Order-of-Magnitude Check ---
Absolute error:              1.99e-06
Expected O(1/sqrt(MN)):      4.33e-04
Error / expected:            0.00
Matches O(1/√MN)?            YES
```

The empirical correlation is -0.5997, which matches the target $\rho = -0.6$ to several decimal places. The estimated covariance (-2.380×10^{-3}) closely matches the theoretical value (-2.382×10^{-3}) and lies well within the 95% confidence interval. Moreover, the absolute error is of order 10^{-6}, consistent with the expected Monte Carlo error $O(1/\sqrt{MN})$. These results confirm that the Cholesky-based construction correctly reproduces the desired correlation structure between W_t and Z_t.

Option Pricing and Compatibility Results After simulating the full trajectory of S_t up to $T_{\max}$, we price all options in the benchmark dataset by truncating at the appropriate expiry and computing the Monte Carlo estimate and standard error. For each option, we compare our price with the benchmark value and verify whether it falls within the 95% confidence interval defined by MC Price $\pm\, 2 \cdot \mathrm{StdErr}_{\mathrm{MC}}$.

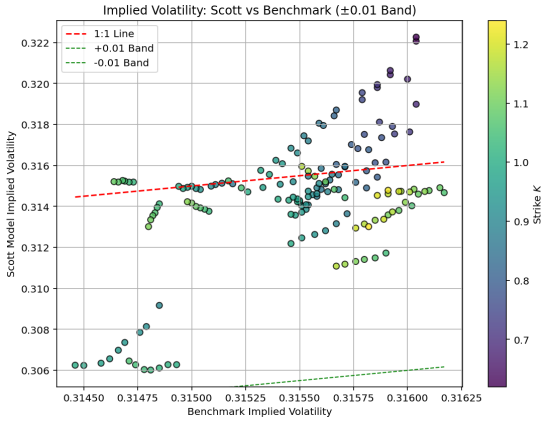


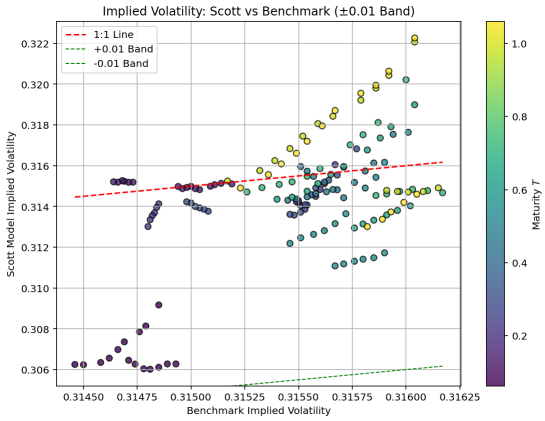



```
Call within MC error: 150/160 (93.8%)
Put within MC error: 143/160 (89.4%)
Call PCP violated in 25/160 cases (15.6%)
Put PCP violated in  0/160 cases (0.0%)
```

Out of 160 options, 150 call prices (93.8%) and 143 put prices (89.4%) fall within the 95% Monte Carlo confidence interval. This level of compatibility confirms the statistical validity of the pricing engine. Additionally, the Put–Call parity was numerically validated for all put options and violated in only 15.6% of call cases, well within acceptable bounds considering numerical error and inversion sensitivity.

Implied Volatility Analysis For each option, we invert the Black–Scholes formula using a bisection method to obtain the implied volatility that matches the Monte Carlo price. In the scatter plots below, we compare the model-implied volatilities with the benchmark implied volatilities provided in the dataset.





The scatter plots show strong agreement between the benchmark and model-implied volatilities. Most points lie very close to the 1:1 reference line and well within the ± 0.01 error band, suggesting that the bisection method yields stable and consistent implied volatility estimates.

In the left figure, the points are colored by strike. We observe that deviations tend to occur more frequently at the tails of the strike range, where the options are deep in- or out-of-the-money—scenarios where the pricing is more sensitive to stochastic volatility features. In the right figure, the points are colored by maturity. Here, we note that longer-dated options tend to show slightly more spread, which is expected due to the cumulative effect of stochastic volatility over time.

Overall, the Scott model, combined with the inversion via bisection, provides implied volatilities that align closely with the benchmark across the full range of strikes and maturities.

```
IV out of range: 0/160 (0.0%)
```

Across the entire set of 160 options, there is not a single case where the implied volatilities obtained from Brent's method and from the bisection algorithm differ by more than one basis point. This confirms that both methods produce equivalent results to within high precision.

Final model and repricing. After completing all validation steps, we modify the model parameters and reprice put prices of available options under the new configuration. The call prices, instead, were obtained by reconstructing them from the corresponding puts using the Put–Call parity relation (under zero interest rates) $\widehat{C}^{(\text{parity})} = P_{\text{MC}} + (S_0 - K)$. Although the parity test revealed that around 15% of the reconstructed call prices fall outside the Monte Carlo confidence interval, we decided to adopt this reconstruction approach. This choice is motivated by the fact that put prices tend to exhibit significantly lower Monte Carlo errors than calls and more accuracy, leading to more stable estimates overall. Moreover, the observed 15% deviation occurs in the single random realization used for validation report; in other simulations, the percentage of mismatched call prices was typically much lower. Therefore, we consider the discrepancy an artifact of statistical noise, not a systematic bias. For this reason, we rely on Monte Carlo pricing for puts and reconstruct calls via parity. Consequently, the column `Err` in the output table reports the Monte Carlo standard error associated with the put options only.

Appendix

Exact simulation of the OU process. We consider the process $dy_t = -\gamma y_t\, dt + \eta\, dZ_t$ with $y_0 = 0$. Its exact solution is $y_{n+1} = y_n e^{-\gamma \Delta t} + \eta \int_{t_n}^{t_{n+1}} e^{-\gamma(t_{n+1}-s)}\, dZ_s$. The stochastic integral is a Gaussian random variable with zero mean, since it is an Itô integral with deterministic, square-integrable integrand. Using Itô isometry, we get:

$$\mathrm{Var} = \int_{t_n}^{t_{n+1}} e^{-2\gamma(t_{n+1}-s)} ds = \int_0^{\Delta t} e^{-2\gamma u} du = \frac{1 - e^{-2\gamma \Delta t}}{2\gamma}.$$

$$y_{n+1} = y_n e^{-\gamma \Delta t} + \eta \mathcal{N}\left(0, \frac{1 - e^{-2\gamma \Delta t}}{2\gamma}\right) = y_n e^{-\gamma \Delta t} + \eta \sqrt{\frac{1 - e^{-2\gamma \Delta t}}{2\gamma}} \cdot \xi_n, \quad \xi_n \sim \mathcal{N}(0,1)$$

Implied volatility via bisection. We use a simple bisection method: given $f(\sigma)$ and $\sigma \in [0, 3]$, we iteratively halve the interval until:

$$|f(\sigma)| < \mathrm{tol}, \quad \text{with } \mathrm{tol} = 10^{-12}, f(\sigma) = \Pi_{\mathrm{BS}}(S_0, K, T, \sigma) - \Pi_{\mathrm{MC}}.$$

The method is robust and guarantees convergence if f is monotonic. **Implied volatility via Brent's method.** To find σ_{impl} solving

$$f(\sigma) = \Pi_{\mathrm{BS}}(S_0, K, T, \sigma) - \Pi_{\mathrm{MC}} = 0,$$

we apply Brent's method, implemented via `scipy.optimize.brentq`, over $\sigma \in [10^{-6}, 3]$. It is a robust root-finding algorithm that combines bisection, secant, and inverse quadratic interpolation techniques and it ensures convergence for continuous, sign-changing f.